UCLouvain

Combinatorial optimization
Homework
Report

Optimal design of an Immunization Campaign

Élèves :
Pierre Straet
Marguerite Lafeuillade

Enseignants :
Julien Hendrickx
Geovani Nunes Grapiglia

24 novembre 2024

Table des matières

1 Modeling the problem **2**
1.1 Variables . . . 2
1.2 Objective and constraints . . . 2
1.2.1 Constraints . . . 2
1.2.2 Objective function . . . 5

2 Relaxation of the problem **5**

3 Improving the model by valid inequalities **5**

4 LP relaxation **6**

5 A first primal bound **6**
5.1 Some properties . . . 6
5.2 Greedy approach . . . 6
5.3 Local search . . . 7

6 Implementation and numerical experiments **8**
6.1 Implementation . . . 8
6.1.1 coord20-5-2_processed.dat . . . 9
6.1.2 coord50-5-1_processed.dat . . . 10
6.1.3 coord100-10-2b_processed.dat . . . 10
6.2 Numerical experiments . . . 11
6.2.1 Greedy Algorithm *VS* Greedy + Local Search . . . 12
6.2.2 OptVax1 *VS* OptVax2 . . . 12
6.2.3 OptVax1 *VS* OptVax2 with warm-start . . . 12
6.2.4 OptVax2-LP . . . 13
6.2.5 General comparison . . . 13

1 Modeling the problem

1.1 Variables

Here we give all variables we will use in the model.

1. Let $x_i \in \{0,1\}$ for $i \in I$ be the variable that decides if the VC i should be open or not.
$$x_i = \begin{cases} 1 & \text{if VC } i \in I \text{ is opened,} \\ 0 & \text{otherwise.} \end{cases}$$
2. Let $y_{ij} \in \{0,1\}$ for $i \in I$ and $j \in J$ be the variable that indicates whether locality j is covered by VC i or not.
$$y_{ij} = \begin{cases} 1 & \text{if the VC } i \in I \text{ is opened and cover the locality } j \in J, \\ 0 & \text{otherwise.} \end{cases}$$
3. Let $v_m \in \{0,1\}$ for $m \in \{1,...,M\}$ be the variable that decides if the MMT m leaves the chosen VC or not (in order to simplify the notation, we define the set $M := \{1,...,M\}$).
$$v_m = \begin{cases} 1 & \text{if MMT } m \in M \text{ is used,} \\ 0 & \text{otherwise.} \end{cases}$$
4. Let $t_{mj} \in \{0,1\}$ for $m \in M$ and $j \in J$ be the variable that indicates whether the locality j is visited by MMT m or not.
$$t_{mj} = \begin{cases} 1 & \text{if the MMT } m \in M \text{ is used and cover the locality } j \in J, \\ 0 & \text{otherwise.} \end{cases}$$
5. Let $w_{mjk} \in \{0,1\}$ for $m \in M$ and $j,k \in I \cup J$ be the variable that decides if the MMT m goes from the location j to location k or not.
$$w_{mjk} = \begin{cases} 1 & \text{if the MMT } m \in M \text{ goes from locality } j \in I \cup J \text{ to } k \in I \cup J, \\ 0 & \text{otherwise.} \end{cases}$$
6. Let $u_{mj} \in \mathbb{N}$ for $m \in M$ and $j \in J$ be the variable that indicates in which position the locality j is on the route of the MMT m.
$$u_{mj} = \begin{cases} n & \text{if the MMT } m \in M \text{ visits the locality } j \in J \text{ in n-th position,} \\ 0 & \text{otherwise.} \end{cases}$$

1.2 Objective and constraints

1.2.1 Constraints

1. One and only one vaccination center must be opened :
$$\sum_{i \in I} x_i = 1.$$

2. A locality can be either ; not visited, visited at most once by a MMT or within the range of an opened VC :

$$\forall j \in J \qquad \sum_{m \in M} t_{mj} + \sum_{i \in I} y_{ij} \leq 1.$$

Explanation : if the left term is equal to 1, it means that the locality is visited by a MMT. In the case where the right term is equal to 1, it indicates that the locality is covered by a MMT. As both term could be 0, the locality may be not visited.

3. Each route corresponds to a mobilized MMT :

$$\forall j \in J \qquad \sum_{m \in M} \sum_{\substack{k \in I \cup J \\ k \neq j}} w_{mjk} \leq 1$$

$$\forall k \in J \qquad \sum_{m \in M} \sum_{\substack{j \in I \cup J \\ j \neq k}} w_{mjk} \leq 1.$$

Explanation : these two equations ensure that the sum of all MMT arriving to or starting from a locality must be at most 1.

$$\forall m \in M, \quad \forall k \in I \cup J \qquad w_{mkk} = 0.$$

Explanation : The constraint forces the MMT to move, i.e. a MMT can't goes to location k to location k.

$$\forall m \in M, \quad \forall k \in I \cup J \qquad \sum_{\substack{j \in I \cup J \\ j \neq k}} w_{mjk} = \sum_{\substack{j \in I \cup J \\ j \neq k}} w_{mkj}.$$

Explanation : the sum of all MMT starting from a location is exactly the sum of MMT arriving to this location.

$$\forall m \in M, \quad \forall j, k \in J, k \neq j \qquad w_{mjk} + u_{mj} \leq u_{mk} + (|I| + |J|)(1 - w_{mjk}).$$

Explanation : this is the sub-turns constraint. It ensures that if a MMT goes from locality j to locality k, the position of k in the route of the MMT m is one more than that of j.

$$\forall m \in M, \quad \forall j \in J \qquad \sum_{k \in I \cup J} w_{mkj} = t_{mj}.$$

Explanation : this constraint ensures that the variable t_{mj} does what we want, i.e. a locality j is visited by a MMT if there is one that arrive at this locality.

4. Each route must respect the capacity constraint :

$$\forall m \in M \qquad \sum_{j \in J} t_{mj} q_j \leq v_m Q.$$

Explanation : the sum over the citizens that a MMT visits, must be lower or equal to the capacity of the vehicle. If this MMT doesn't leave the chosen VC, then the sum must be equal to 0.

5. The capacity for the VC is only for specifically conditioned doses delivered by MMTs :
$$\forall i \in I \qquad x_i \sum_{m \in M} \sum_{j \in J} t_{mj} q_j \leq C_i.$$
Explanation : the sum over the citizens visited by all MMTs must be lower or equal to the capacity constraint of the chosen VC. Notice that the factor x_i ensure that we are just taking the chosen VC into account, indeed, for an other VC the constraint will always be valid.

6. The starting point and the end point are the same for each route and turns out to be the chosen VC :
$$\forall i \in I \qquad \sum_{m \in M} \sum_{j \in I \cup J} w_{mij} = x_i \sum_{m \in M} v_m$$
$$\forall i \in I \qquad \sum_{m \in M} \sum_{j \in I \cup J} w_{mji} = x_i \sum_{m \in M} v_m.$$
Explanation : these two constraints ensure that the sum of MMTs arriving/leaving the chosen VC is exactly the sum of all used MMTs. The factor x_i forces to not have MMT arriving/leaving an other VC.

7. All prioritized localities have to be covered :
$$\forall j \in \hat{J} \qquad \sum_{m \in M} t_{mj} + \sum_{i \in I} y_{ij} = 1.$$
Explanation : forces all prioritized localities to be either visited by a MMT or covered by the chosen VC.

8. *Additional constraints.*
   - Budget constraint :
$$\sum_{i \in I} x_i f_i + \sum_{m \in M} v_m p + \sum_{m \in M} \sum_{k \in I \cup J} \sum_{j \in I \cup J} d_{ij} w_{mjk} \leq B$$
Explanation : for the budget constraint, we have to sum the cost of opening the chosen VC, the cost of all routes opened and the cost of distance travelled.

   - Ensuring that the variable y_{ij} does what we want :
$$\forall i \in I, \quad \forall j \in J \qquad y_{ij}(d_{ij} - r_i)x_i \leq 0 \tag{1}$$
$$\forall i \in I, \quad \forall j \in J \qquad (1 - y_{ij})(r_i - d_{ij})x_i \leq 0 \tag{2}$$
$$\forall i \in I, \quad \forall j \in J \qquad y_{ij}(1 - x_i) = 0. \tag{3}$$
Explanation : first notice that the factor x_i in (1) and (2) implies that we are just taking the chosen VC into account, since for the other, the inequalities will always be valid. Thus (1) ensures y_{ij} to be 0 if $r_i < d_{ij}$, meaning that locality j is not in the radius of the VC. (2) does the same for the inverse case. Finally, (3) forces all y_{ij} to 0 if the VC i is not chosen.

1.2.2 Objective function

Since we want to optimize the total number of citizens vaccinated, we will have the following objective function :

$$\text{Maximize} \sum_{j \in J} q_j \left(\sum_{m \in M} t_{mj} + \sum_{i \in I} y_{ij} \right).$$

2 Relaxation of the problem

By fixing the location of the Vaccination Center and assigning a subset of localities to each Mobile Medical Team (MMT), the problem reduces to a form of Travelling Salesman Problem (TSP). For each MMT, we need to find the optimal route that visits the assigned localities.

Therefore, the relaxed problem can be understood as a multi-vehicle TSP with a fixed point of departure and arrival (the VC). Each MMT's route is a solution to a TSP, and the objective is to minimize the total travel cost.

In cases, where there is no subset of localities assigned to the MMTs, the final objective of the problem is to maximize the quantity q of vaccinated people (respecting the budget, and the other constraints). Therefore, some routes may not be the shortest possible, as they will prioritize maximizing q instead. As we shall see later, if we adjust the budget, particularly by introducing C_{km}, a travel cost per kilometre greater than 1, we can force the routes to be shorter and still keep the objective of maximizing the quantity q of vaccinated people.

The constraint matrix in this relaxed problem is not totally unimodular. This problem still involves capacity/budget constraints, and can't be composed of just {-1,0,1} values, which is a necessary condition to be TU.

3 Improving the model by valid inequalities

We want a lower bound on the number of vehicles to be used, the idea we will express is that Q times the number of MMTs must be greater or equals to the total number of vaccinated people. Obviously, if this was smaller, this means that at least one MMT exceed his capacity constraint. We thus have the following additional constraint :

$$\sum_{m \in M} v_m \geq \frac{1}{Q} \sum_{j \in J} q_j \left(\sum_{m \in M} t_{mj} \right).$$

Hence the inequality gives us a lower bound for the number of MMTs deployed. Theoretically, adding this constraint won't change the optimal solution of the model. Still, it is not useless. Adding constraint permit to reduce the feasible space of our model, especially for the relaxation of the problem (see next section), and thus sometimes, it helps the solver to find the best solution.

4 LP relaxation

The LP relaxation is a technique use to find a upper bound on our problem. As it involves the suppression of the integer constraints of all variable, it may permit the solver to improve the optimal solution. In our case, deploying half a car becomes a feasible solution for example.

The changes are therefore only on the variables which becomes :

- $x_i \in [0,1]$ for $i \in I$;
- $y_{ij} \in [0,1]$ for $i \in I$ and $j \in J$;
- $v_m \in [0,1]$ for $m \in M$;
- $w_{mjk} \in [0,1]$ for $m \in M$ and $j, k \in I \cup J$;
- $u_{mj} \in \mathbb{R}$ for $m \in M$ and $j \in J$;

where each one is define the same than in the subsection *variables*.

5 A first primal bound

5.1 Some properties

Let X be the vector stacking all the decision variables and $z(X)$ the objective function .

Let $z^* = \max_{X \in F} z(X)$, the optimal value of the problem. Define,

$$z_{max}^{FVC}(X) = \max_{i \in I} \{z(X) \mid \text{ s.t. VC is opened at location } i \text{ in } X\}$$

We want to find a necessary and sufficient condition on X such that the following equation holds :

$$z_{max}^{FVC}(X) = z^*$$

The condition is that there exists a VC i, and a feasible solution X^* where $z(X^*) = z^*$. This solution is necessarily optimal compared to the other solutions obtained for the other VCs. We will then have $z_{max}^{FVC}(X) = \max_{i \in I} \{z(X)\} = z(X^*)$ and so $z_{max}^{FVC}(X) = z^*$

5.2 Greedy approach

If we suppose that one (and only one) VC is fixed. The greedy algorithm for this problem consists to create a first MMT that starts from this fixed VC, and visits the nearest locality that is not within the VC's range. After visiting this first locality, the MMT go to the next nearest locality, continuing in this way until no more localities can be visited without violating the constraints. Indeed, before visiting the next localities, we check if it respects the constraints of capacity Q, , the capacity $C[VC_{chosen}]$, and the budget B. For the budget constraint, we need to verify that there is enough remaining budget to travel to the next locality and return to the VC, in the case the next locality is the last one visited by the current MMT. When these constraints are not respect for the current MMT, if $B_{remaining} > p$ and the constraint of maximum M MMTs is still respected, we open a new MMT, proceed as before, and continue until we are blocked by one or more

constraints.

Once the greedy solution is constructed, we verify if all high-priority localities (that are not in the VC's range) have been visited. If any high-priority locality is missing, we restart at 0 with now a focus on visiting the high-priority localities first.

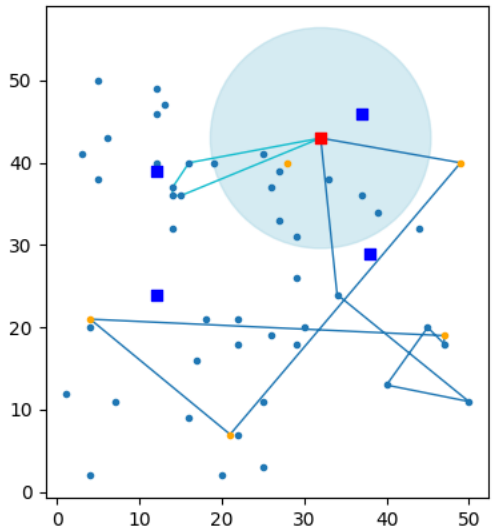
If no prior information is available about the fixed VC, we can extend the greedy approach by considering all possible VC. For each VC, we apply the greedy approach and the best solution (i.e. the solution where the most people are vaccinated) across all possible VC is then selected. This greedy approach provides a primal bound for the problem.

5.3 Local search

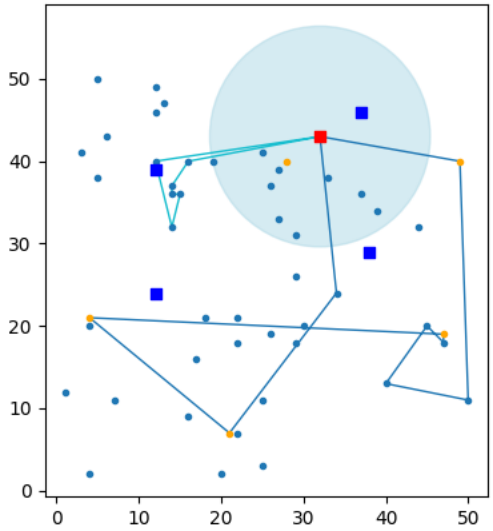
The local search algorithm starts with the greedy solution and iteratively improves it by exploring neighbour solutions.

The local search algorithm starts by calculating the total distance of the given route obtained after the greedy approach. For each route, the algorithm tries to improve the solution by reversing segments of the route and recalculating the total distance. If this change gives a shorter distance of the route, the new route is kept as the best solution. This process is repeated for all routes, and the algorithm returns a list of the improved routes.

After this search, we return to the greedy approach, where we keep the improved routes (except the last one) and try to improve the last one. Indeed, the last route constructed with the greedy algorithm is often constrained by budget (or the capacity C), unlike the other routes, which are constrained by the capacity Q. Thus, the budget saved by the shorter distance of the routes from $[0, m-1]$ can sometimes allow the last route to be extended.



Greedy approach



Local search amelioration

The example above is inspire from coord50-5-3b_processed.dat, **but with changes of the parameters $C_{\mathbf{km}}$ (cost per kilometer greater than 1) to provide a simple and clear illustration**. Thanks to the local search, the MMTs can visit an other locality, increasing the total number of vaccinated people from 362 to 389.

6 Implementation and numerical experiments

6.1 Implementation

We have realized an implementation of :

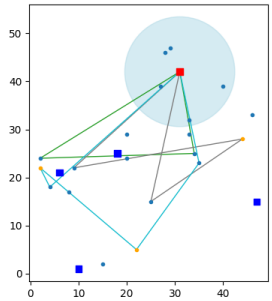
- OptVax1
- OptVax2
- OptVax2-LP
- Greedy algorithm and local search

As well as a solution visualizer (The open VC is in red, its range is represented by a blue disk, and the prioritized localities are in orange).

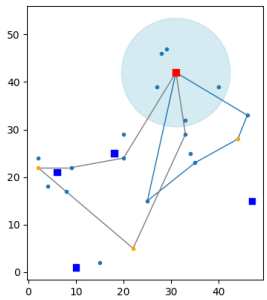
As explained before, the objective is to maximize the number of people that can be fully vaccinated. For the illustration we have sometimes reduced the budget, by introducing C_{km}, a travel cost per kilometre greater than 1, to force the routes to be shorter just for a better visualisation (ensuring that the budget did not decrease enough to impact the number of people vaccinated).

Here are some examples of the results obtained :

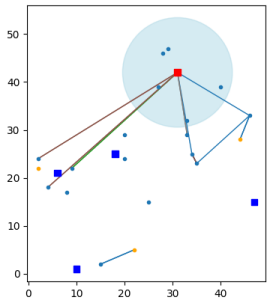
6.1.1 coord20-5-2_processed.dat



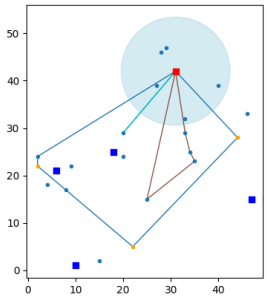
OptVax1
$C_{km} = 1$



OptVax1
$C_{km} = 12$



OptVax2 LP
-



Local
search

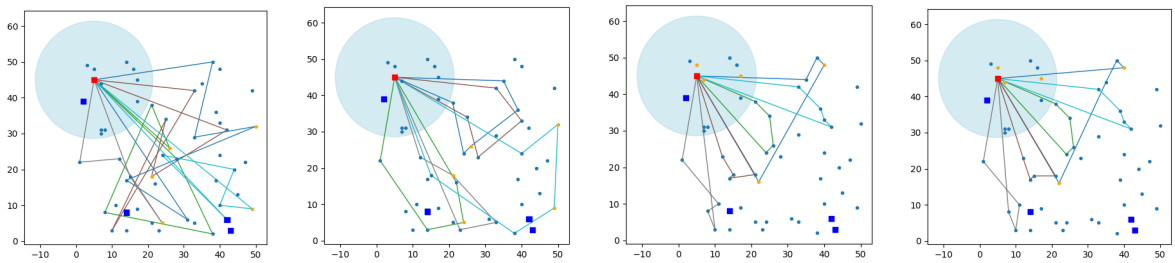
For this example, the optimization by OptVax1 gives a result of 216 vaccinated people, while the Greedy approach gives a result of 207 vaccinated people. Local search does not improve the routes for this instance.

The optimization by OptVax2 does not change the optimal value obtained by OptVax1. The interest of OptVax2 implementation is mostly related to CPU time, as it will be discussed later.

As it can be seen by the visualization and as we knew, OptVax2_LP does not give a feasible solution but an upper bound for this problem.

A last thing that is important to discuss is the difference between the two first pictures. In the first one, we can see that the routes the solver found seem not the best one. This is due to the fact that it has enough budget to permit it, and thus that the tight constraint is something else. By increasing the cost of travelling ($C_{km} = 12$) we could force the budget to be more restrictive and thus leading to better routes to save the more money as possible.

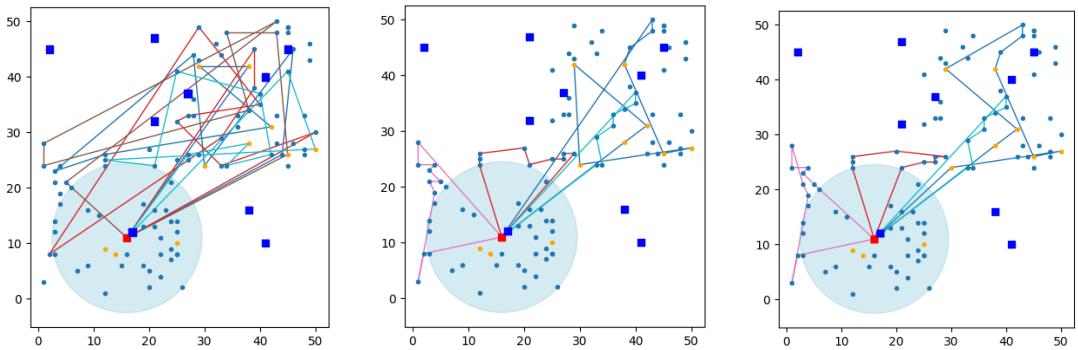
6.1.2 coord50-5-1_processed.dat



OptVax1 $C_{km} = 1$ | OptVax1 $C_{km} = 8$ | Greedy approach | Local search

For this example, the optimization by OptVax1 gives a result of 497 vaccinated people, while the Greedy approach and local approach gives a result of 457 vaccinated people. The local search algorithm improves the routes by reducing distances but not enough to allow the latest road to reach new locations.

6.1.3 coord100-10-2b_processed.dat



OptVax1 $C_{km} = 1$ | Greedy approach | Local search

For this example, the optimization by OptVax1 gives a result of 1107 vaccinated people, while the Greedy approach and local approach gives a result of 1103 vaccinated people. The local search algorithm improves the routes by reducing distances but not enough to allow the latest road to reach new locations.

Increasing the value C_{km} also increases the complexity and the running time. For this more complex example , we stay at $C_{km} = 1$. Even if it doesn't give the prettiest of visualizations

6.2 Numerical experiments

We begin by giving our results. Unfortunately, we was not able to compute a solution for instances of size 200, even not a feasible result. Also, for some instances of size 100 the solver couldn't find a optimal solution before the time out was reached, this one was fixed at $30min$. The later implies that the solution for two different models may differ.

Greedy		**Local Search**		**OptVax1**		**OptVax2**		**OptVax2 - LP**	
Sol.	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**
256	0.0	256	0.016	265	1.25	265	1.25	265	2.84375
308	0.0	308	0.0	308	0.765625	308	0.765625	308	1.15625
207	0.0	207	0.0	216	1.125	216	1.3125	216	2.46875
302	0.0	302	0.0	302	0.640625	302	0.703125	302	2.09375

Table 1 – Results for instances of size 20.

Greedy		**Local Search**		**OptVax1**		**OptVax2**		**OptVax2 - LP**	
Sol.	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**
457	0.0	457	0.0	497	36.671875	497	79.84375	497	6.71875
530	0.0	530	0.0	565	414.296875	537	time out	565	16.53125
482	0.0	482	0.0	510	20.71875	510	37.4375	519	10.9375
602	0.0	602	0.0	616	9.578125	616	9.3125	616	20.734375
683	0.0	683	0.0	686	11.8125	686	10.5	686	12.84375
571	0.0	571	0.0	581	4.8125	581	5.75	640	11.140625
460	0.0	460	0.0	504	15.046875	504	25.0	504	6.203125
564	0.0	564	0.0	574	14.4375	574	11.84375	578	20.34375

Table 2 – Results for instances of size 50.

Greedy		Local Search		OptVax1		OptVax2		OptVax2 - LP	
Sol.	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**
721	0.0	721	0.0	815	time out	815	time out	851	37.25
948	0.0	948	0.016	962	1685.8281	962	1471.5469	962	532.8281
974	0.0	974	0.0	1014	121.6875	1014	246.1094	1045	36.875
1225	0.016	1225	0.0	1264	271.8125	1264	489.875	1340	36.6406
709	0.0	709	0.016	738	301.6406	738	312.4063	795	37.3906
823	0.0	823	0.016	838	time out	831	time out	1004	185.40625
796	0.016	796	0.016	765	time out	800	time out	844	319.9688
840	0.016	840	0.016	844	405.7813	844	365.75	844	802.6563
1063	0.0	1063	0.0	1063	132.875	1063	457.5	1063	218.6719
1103	0.0	1103	0.016	1107	294.625	1107	367.29688	1107	541.125
833	0.016	833	0.0	837	463.5156	837	1937.1719	837	331.0156
831	0.016	831	0.0	849	282.7031	849	444.9219	889	933.2656

TABLE 3 – Results for instances of size 100.

6.2.1 Greedy Algorithm *VS* Greedy + Local Search

First we compare the Greedy Algorithm with the Local Search. One notice few differences between both heuristic. They always give the same solution since the improvement of the Greedy Algorithm by the Local Search couldn't be exploit to add a locality to one path. There is only the CPU time that differs, but not significantly.

6.2.2 OptVax1 *VS* OptVax2

If we ignore the instances where the solver time out, we notice that the solution is always the same, meaning that the model is invariant. In contrast, the CPU time may differ between both model. Surprisingly, the valid constraint we added in the OptVax2 model seems to slow down the solver most of the time instead of helping it. We even have a time out for this one while OptVax1 find an optimal solution.

Using a test statistic on a given metric : $\sigma'_{l,k} = \frac{\tilde{z}_{l,k}}{\tau_{l,k}}$, we chose the *Wilcoxon-Mann-Whitney* one, we get a p-value of 0.672, thanks to scipy.stat, meaning that both distributions differ slightly indicating a robust result.

6.2.3 OptVax1 *VS* OptVax2 with warm-start

We give here the results for the methods run with a warm start :

The warm start does not seem to improve the CPU time but even increase it a little bit, which is surprising since it should help the solver when it is looking for a first feasible solution. However, in other optimization problem, a warm start is sometimes necessary.

OptVax1		OptVax2		OptVax2 - LP	
Sol.	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**
265	1.296875	265	1.203125	265	2.765625
308	0.765625	308	0.78125	308	1.1875
216	1.109375	216	1.296875	216	2.453125
302	0.609375	302	0.703125	302	1.953125

TABLE 4 – Results for instances of size 20 with OptVax methods and a warm start.

OptVax1		OptVax2		OptVax2 - LP	
Sol.	**CPU**	**Sol.**	**CPU**	**Sol.**	**CPU**
497	36.75	497	83.0	497	7.375
565	410.59375	537	844.671875	565	17.3125
510	21.296875	510	40.265625	519	11.015625
616	9.578125	616	9.46875	616	24.015625
686	12.8125	686	11.6875	686	13.09375
581	4.9375	581	5.9375	640	11.140625
504	16.203125	504	26.6875	504	6.984375
574	15.296875	574	12.640625	578	22.09375

TABLE 5 – Results for instances of size 50 with OptVax methods and a warm start.

Moreover, if we reiterate the test statistic already use in the later, we find a p-value of 0.8 this time, which still indicates a great robustness.

6.2.4 OptVax2-LP

For the OptVax2-LP model, we have very interesting results. Indeed, a lot of instances have the same optimal value for the relax problem and the others. Reassuring us that the solution is optimal for the other model since it is easier to compute the solution of a non-integer problem. For all instances were the optimal solution found differ from the relax problem, it always do slightly. Also, the CPU time for solving a relax problem is much smaller than others (as we have just said, it is easier to compute so it is not surprising).

6.2.5 General comparison

Now, we will compare each model together. We first notice that unexpectedly, the heuristic such as the Greedy Algorithm or the Local Search are not too bad, they are not that far from the optimal solution. It is much more surprising since the solution given by the Local Search do not thus help the solver. To resume, we could see in the results we get that heuristic methods are much more faster than those provided by Gurobi, but even

if the solution found is not that bad, Gurobi often find the optimal solution which is our ultimate goal. Finally, if we had to chose a model for solving our problem, we will take the OptVax1 model, since the OptVax2 bring no value and can be slower. Furthermore, if one is satisfy by a quite good lower bound, the Local Search would be the best choice, and for an upper bound, choosing the OptVax2-LP model would be a good idea.

Even if our first model OptVax1 seems to be the best here, this project has familiarized us with other models which can be very useful in other contexts. Valid constraints (for OptVax2) and warm start could be very help full sometimes if it permits the solver to considerably reduce the feasible domain or to start from a solution very close to the optimal solution.